# gemm_dma_engine

## DMA hai kênh: đọc và ghi độc lập

Port list, bộ sinh địa chỉ 2D, cắt burst, kênh đọc, kênh ghi, lỗi và abort

Bộ tăng tốc GEMM INT8 trên Chipyard SoC

| | |
|---|---|
| **Module** | `gemm_dma_engine` (instance `u_dma` trong `gemm_accel_top`) |
| **Module con** | `gemm_dma_agen` (2 instance), `gemm_dma_rd`, `gemm_dma_wr`, `gemm_dma_wfifo` |
| **Cấu hình** | Bản đầy đủ: lệnh FETCH, stride, hai kênh chạy đồng thời (BS-04). Ở bản cơ bản FSM không bao giờ phát hai lệnh cùng lúc và mọi stride bằng độ dài hàng; module không cần tham số riêng |
| **Tuân theo** | Spec hệ thống v1.0: mục 2.3 (DAT-01 … DAT-05), mục 5.3 (DMA-01 … DMA-10), mục 5.4 (SPM-02, SPM-05), mục 6 (port `m_axi_*`), mục 7 (DSC-01, DSC-09, PP-01) |
| **Kèm theo** | `gemm_dma_model.py`: golden cắt burst và địa chỉ; mô hình hai kênh với slave AXI ngẫu nhiên (trễ, lỗi RESP, `rlast` sai, abort) |

# Mục lục

1 Chức năng 3

2 Tham số 3

3 Port list 3

4 Vi kiến trúc 5

5 `gemm_dma_agen`: địa chỉ và burst 5

6 `gemm_dma_rd`: kênh đọc 6

7 `gemm_dma_wr`: kênh ghi 7

8 Reset, hiện thực, kiểm chứng 8

# 1 Chức năng

DMA chép dữ liệu giữa DRAM và scratchpad theo lệnh của Main FSM. **Kênh đọc** chạy FETCH, LOAD_A, LOAD_B, LOAD_BIAS: đọc DRAM bằng burst AXI và ghi vào SRAM (hoặc trả 16 word descriptor cho FSM). **Kênh ghi** chạy STORE_C: đọc buffer C và ghi DRAM. Hai kênh độc lập hoàn toàn, mỗi kênh tối đa một burst outstanding. Mỗi lệnh là một bản sao 2D: `rows` hàng, mỗi hàng `rwords` word, các hàng cách nhau `stride` byte.

**Bảng 1.1:** Chức năng

| ID | Yêu cầu |
|---|---|
| DMA-F01 | Với lệnh (base, rows, rwords, stride), word thứ n ($0 \le n < \text{rows} \cdot \text{rwords}$), r = n / rwords, w = n mod rwords, nằm ở địa chỉ DRAM base + r · stride + 4w và ở địa chỉ scratchpad theo bảng 5.2. Mỗi word được chép đúng một lần, theo thứ tự n tăng dần |
| DMA-F02 | Mỗi lệnh kết thúc bằng đúng một xung `o_*_done` hoặc `o_*_error` (DMA-01). Lỗi: `rresp` hoặc `bresp` $\neq$ OKAY, hoặc `rlast` sai vị trí (DMA-08) |
| DMA-F03 | DMA không đảo byte, không đổi dữ liệu (DMA-02); không biết M, N, K hay ý nghĩa của buffer |
| DMA-F04 | Kênh đọc và kênh ghi được phép có burst AXI cùng lúc (PP-01). AXI4 cho phép hai kênh vì chúng không chia sẻ tín hiệu |

# 2 Tham số

**Bảng 2.1:** Tham số

| Tham số | Mặc định | Ý nghĩa |
|---|---|---|
| `MAX_BURST_BEATS` | 16 | Số beat tối đa mỗi burst, 1 … 256. Nối từ tham số cùng tên của top |
| `WFIFO_DEPTH` | 4 | localparam: số word của FIFO dữ liệu ghi. 4 đủ cho 1 beat W mỗi chu kỳ vì SRAM trả dữ liệu sau đúng 1 chu kỳ |

# 3 Port list

**Bảng 3.1:** Port nội bộ của `gemm_dma_engine`

| Port | Rộng | Chiều | Nối với | Mô tả |
|---|---|---|---|---|
| *Clock, reset* | | | | |
| `clk`, `rst_n`, `i_soft_clr` | $1 \times 3$ | In | top, FSM | |
| *Lệnh kênh đọc* | | | | |
| `i_rd_start` | 1 | In | FSM | Xung bắt đầu; các `i_rd_*` tham số hợp lệ ở chu kỳ này |
| `i_rd_dst` | 2 | In | FSM | 0: A, 1: B, 2: BIAS, 3: descriptor |
| `i_rd_base` | 32 | In | FSM | Địa chỉ byte hàng đầu, căn 4 |
| `i_rd_rows`, `i_rd_rwords` | 6, 6 | In | FSM | 1 … 32 mỗi trường |
| `i_rd_stride` | 16 | In | FSM | Bội của 4, $\ge 4 \cdot$ `rwords` |
| `i_rd_bank` | 1 | In | FSM | Bank scratchpad đích |
| `i_rd_abort` | 1 | In | FSM | Mức: abort (DMA-10) |
| `o_rd_busy` | 1 | Out | FSM | Từ chu kỳ sau `i_rd_start` tới hết chu kỳ có done/error |
| `o_rd_done`, `o_rd_error` | 1, 1 | Out | FSM | Xung 1 chu kỳ |

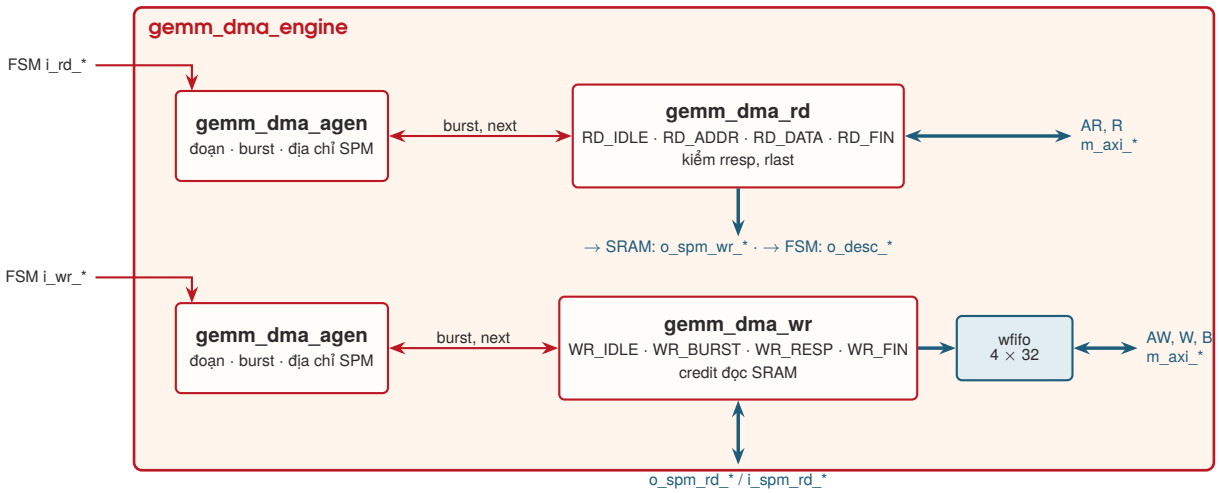
**Bảng 3.1:** Port nội bộ của `gemm_dma_engine` (tiếp theo)

| Port | Rộng | Chiều | Nối với | Mô tả |
|---|---|---|---|---|
| `o_desc_valid`, `o_desc_idx`, `o_desc_data` | 1, 4, 32 | Out | FSM | Một word descriptor (chỉ khi dst = 3) |
| *Lệnh kênh ghi* | | | | |
| `i_wr_start`, `i_wr_fmt` | 1, 1 | In | FSM | Xung bắt đầu; 0: INT32, 1: INT8 |
| `i_wr_base`, `i_wr_rows`, `i_wr_rwords`, `i_wr_stride`, `i_wr_bank` | 32, 6, 6, 16, 1 | In | FSM | Như kênh đọc |
| `i_wr_abort` | 1 | In | FSM | |
| `o_wr_busy`, `o_wr_done`, `o_wr_error` | $1 \times 3$ | Out | FSM | |
| *Cổng ghi scratchpad (A, B, BIAS)* | | | | |
| `o_spm_wr_valid` | 1 | Out | SRAM | Ghi một word |
| `o_spm_wr_buf` | 2 | Out | SRAM | 0: A, 1: B, 2: BIAS |
| `o_spm_wr_bank` | 1 | Out | SRAM | |
| `o_spm_wr_addr` | 10 | Out | SRAM | Địa chỉ word logic (SPM-02) |
| `o_spm_wr_data` | 32 | Out | SRAM | |
| `i_spm_wr_ready` | 1 | In | SRAM | Bắt tay; SRAM luôn giữ bằng 1 (SRM-I01) |
| *Cổng đọc scratchpad (C)* | | | | |
| `o_spm_rd_valid`, `o_spm_rd_bank`, `o_spm_rd_addr` | 1, 1, 10 | Out | SRAM | Yêu cầu đọc một word C |
| `i_spm_rd_ready` | 1 | In | SRAM | Luôn bằng 1 |
| `i_spm_rd_rvalid`, `i_spm_rd_rdata` | 1, 32 | In | SRAM | Dữ liệu, đúng 1 chu kỳ sau yêu cầu được nhận |
| *AXI4-Full master: 25 port `m_axi_*` của bảng 6-3 spec hệ thống, nối thẳng ra top* | | | | |

**Bảng 3.2:** Giao tiếp

| ID | Yêu cầu |
|---|---|
| DMA-I01 | `i_rd_start` chỉ đến khi `o_rd_busy` = 0 (FSM-I02). DMA chốt mọi tham số ở chu kỳ đó; FSM được đổi chúng ngay sau |
| DMA-I02 | `i_rd_abort`, `i_wr_abort` được chốt thành cờ `abort_seen` trong suốt lệnh; xóa khi lệnh kết thúc |
| DMA-I03 | `m_axi_arsize` = `m_axi_awsize` = 3'b010; `m_axi_arburst` = `m_axi_awburst` = 2'b01; `m_axi_wstrb` = 4'b1111; `m_axi_bready` = 1 (DMA-04, DMA-07) |
| DMA-I04 | Mọi port `m_axi_*` ra lấy từ thanh ghi, trừ `m_axi_rready` (= trạng thái $\wedge$ `i_spm_wr_ready`); không có đường tổ hợp từ `m_axi_*` vào tới `m_axi_*` ra (TOP-04) |
| DMA-I05 | `o_spm_wr_valid` = `m_axi_rvalid` $\wedge$ `m_axi_rready` $\wedge$ (beat nằm trong burst) $\wedge$ (dst $\neq$ 3); `o_spm_wr_data` = `m_axi_rdata`. Đường tổ hợp này chỉ đi vào SRAM, được phép |

# 4 Vi kiến trúc




**Hình 4.1:** Cấu trúc bên trong `gemm_dma_engine`: hai kênh giống nhau về cách sinh địa chỉ, không chia sẻ trạng thái

**Bảng 4.1:** Module con

| Module | Instance | Vai trò |
|---|---|---|
| `gemm_dma_agen` | `u_agen_rd`, `u_agen_wr` | Chia lệnh thành đoạn liên tục, cắt burst, đếm (r, w) và sinh địa chỉ scratchpad |
| `gemm_dma_rd` | `u_rd` | Kênh đọc: AR, R; ghi SRAM hoặc xuất descriptor; lỗi, abort |
| `gemm_dma_wr` | `u_wr` | Kênh ghi: đọc SRAM theo credit, AW, W, B; lỗi, abort |
| `gemm_dma_wfifo` | `u_wfifo` | FIFO đồng bộ `WFIFO_DEPTH` × 32, đầu ra có thanh ghi (head) |

# 5 `gemm_dma_agen`: địa chỉ và burst

**Bảng 5.1:** Port của `gemm_dma_agen`

| Port | Rộng | Chiều | Mô tả |
|---|---|---|---|
| `i_load` | 1 | In | Chốt lệnh mới (cùng chu kỳ `i_*_start`) |
| `i_base`, `i_rows`, `i_rwords`, `i_stride` | 32, 6, 6, 16 | In | Tham số lệnh |
| `i_wide` | 1 | In | 1: địa chỉ SPM {r[4:0], w[4:0]}; 0: {r[4:0], w[2:0]} |
| `o_has_burst` | 1 | Out | Còn burst chưa phát |
| `o_burst_addr`, `o_burst_beats` | 32, 9 | Out | Burst kế tiếp: địa chỉ byte và số beat (1 … 256); thanh ghi |
| `i_burst_next` | 1 | In | Burst hiện tại đã được phát; tính burst sau |
| `i_word_next` | 1 | In | Một word đã được chép; tăng (r, w) |
| `o_spm_addr`, `o_word_idx` | 10, 4 | Out | Địa chỉ SPM của word hiện tại; n[3:0] (cho `o_desc_idx`) |

**Bảng 5.2:** Địa chỉ scratchpad theo đích (SPM-02)

| Lệnh | i_wide | Địa chỉ word n (r = hàng, w = word trong hàng) |
|---|---|---|
| LOAD_A, LOAD_B | 0 | {r[4:0], w[2:0]} = {i, kw} hoặc {k, jw} |
| LOAD_BIAS | 1 | {0, w[4:0]} = j |

**Bảng 5.2:** Địa chỉ scratchpad theo đích (SPM-02) (tiếp theo)

| Lệnh | i_wide | Địa chỉ word n (r = hàng, w = word trong hàng) |
|---|---|---|
| STORE_C, INT32 | 1 | {r[4:0], w[4:0]} = {i, j} |
| STORE_C, INT8 | 0 | {r[4:0], w[2:0]} = {i, jw} |
| FETCH | — | Không dùng SPM; o_word_idx = w (0 … 15) |

**Bảng 5.3:** Chia đoạn và cắt burst

| ID | Yêu cầu |
|---|---|
| AG-01 | Lệnh liên tục khi stride = 4 · rwords: một đoạn dài rows · rwords word bắt đầu ở base. Ngược lại: rows đoạn, đoạn r dài rwords word bắt đầu ở base + r · stride (DSC-09) |
| AG-02 | Trong một đoạn, tại địa chỉ a, burst có min(số word còn lại của đoạn, MAX_BURST_BEATS, $(4096 - a \bmod 4096)/4$) beat (DMA-03). Burst không bao giờ vượt biên 4 KiB và không bao giờ trải qua khoảng hở giữa hai đoạn |
| AG-03 | o_burst_* là thanh ghi, hợp lệ từ chu kỳ sau i_load và từ chu kỳ sau i_burst_next. Địa chỉ đoạn tính bằng cộng dồn stride (không dùng bộ nhân 32 bit); độ dài đoạn liên tục rows · rwords $\leq$ 1024 cần bộ nhân 6 $\times$ 6 hoặc cộng dồn |
| AG-04 | Bộ đếm (r, w) độc lập với burst: i_word_next tăng w; khi w = rwords − 1 thì w $\leftarrow$ 0, r $\leftarrow$ r + 1 |

**VÍ DỤ CẮT BURST**

LOAD_A với M = 32, K = 32 liên tục (lda = 32): một đoạn 256 word; với MAX_BURST_BEATS = 16 và base căn 4 KiB: 16 burst 16 beat. Cùng lệnh với lda = 48: 32 đoạn 8 word, mỗi đoạn một burst 8 beat (nếu không vượt 4 KiB), khoảng hở 16 byte giữa hai hàng không bị đọc. Base = 0x0FF8, một đoạn 256 word: burst đầu 2 beat (tới biên 4 KiB), sau đó 15 burst 16 beat và một burst 14 beat.

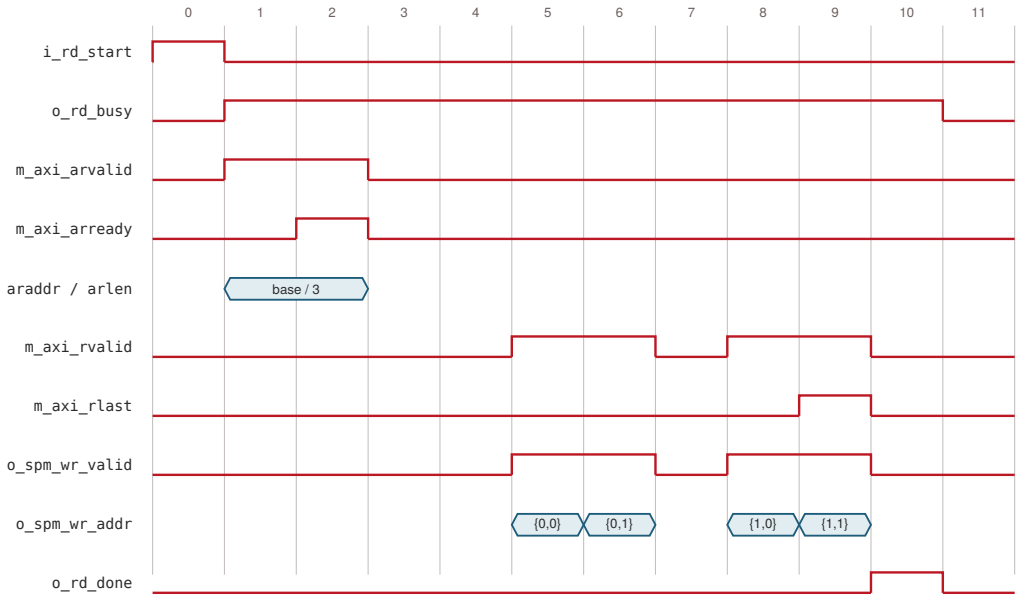
# 6 gemm_dma_rd: kênh đọc

**Bảng 6.1:** Trạng thái kênh đọc

| Trạng thái | Làm gì | Sang |
|---|---|---|
| RD_IDLE | o_rd_busy = 0. i_rd_start: chốt dst, bank; agen.i_load; err $\leftarrow$ 0, abort_seen $\leftarrow$ 0 | RD_ADDR |
| RD_ADDR | arvalid = 1, araddr = o_burst_addr, arlen = beats − 1, giữ tới khi arready; exp $\leftarrow$ beats, got $\leftarrow$ 0 | RD_DATA sau bắt tay AR (agen.i_burst_next) |
| RD_DATA | rready = i_spm_wr_ready (dst = 3: 1). Mỗi beat: got++; nếu got $\leq$ exp: ghi SRAM hoặc xuất descriptor, agen.i_word_next; rresp $\neq$ 0 $\rightarrow$ err $\leftarrow$ 1. Xử lý rlast theo RDC-02 | Khi burst kết thúc: RD_ADDR nếu còn burst và ¬err $\wedge$ ¬abort_seen; ngược lại RD_FIN |
| RD_FIN | Phát o_rd_error nếu err, ngược lại o_rd_done | RD_IDLE |

**Bảng 6.2:** Kênh đọc

| ID | Yêu cầu |
|---|---|
| RDC-01 | Tối đa 1 burst outstanding: AR kế tiếp chỉ được phát sau beat cuối của burst trước (DMA-06) |
| RDC-02 | rlast = 1 ở beat có got < exp: lỗi, burst kết thúc. Beat có got = exp mà rlast = 0: lỗi, tiếp tục nhận (rready = 1) và bỏ mọi beat tới khi rlast = 1 (DMA-08) |
| RDC-03 | Lỗi hoặc abort: không phát AR mới; burst đã phát AR được nhận hết (DMA-07, DMA-09, DMA-10). Abort không đổi kết quả: lệnh bị abort kết thúc bằng done nếu không có lỗi |
| RDC-04 | Dữ liệu beat lỗi vẫn được ghi vào SRAM (DMA-09 cho phép); FSM không dùng kết quả của lệnh lỗi |
| RDC-05 | Thông lượng: mỗi chu kỳ một beat khi slave không chèn bọt; giữa hai burst có đúng 1 chu kỳ RD_ADDR cộng độ trễ AR $\rightarrow$ R của slave |




**Hình 6.1:** LOAD_A với M = 2, K = 8 (rwords = 2, liên tục): một burst 4 beat; slave nhận AR ở chu kỳ 2, trả beat ở 5, 6, 8, 9; `o_rd_done` ở chu kỳ 10

# 7 `gemm_dma_wr`: kênh ghi

**Bảng 7.1:** Trạng thái kênh ghi

| Trạng thái | Làm gì | Sang |
|---|---|---|
| WR_IDLE | `i_wr_start`: chốt fmt, bank; `agen.i_load` với `i_wide` = (fmt = 0); `err`, `abort_seen` ← 0 | WR_NEXT |
| WR_NEXT | 1 chu kỳ: nếu `err` ∨ `abort_seen` ∨ ¬`o_has_burst` thì sang WR_FIN. Ngược lại nạp `aw_pend` ← 1, `w_left` ← beats, `rd_left` ← beats; `agen.i_burst_next` | WR_BURST |
| WR_BURST | Ba việc song song: (1) `awvalid` = `aw_pend` tới khi `awready`; (2) đọc SRAM: `o_spm_rd_valid` = (`rd_left` > 0) ∧ (`fifo_cnt` + `inflight` < 4); mỗi yêu cầu được nhận: `rd_left--`, `agen.i_word_next`; (3) `wvalid` = FIFO khác rỗng ∧ `w_left` > 0, `wdata` = đầu FIFO, `wlast` = (`w_left` = 1); mỗi bắt tay W: lấy khỏi FIFO, `w_left--` | WR_RESP khi AW xong và `w_left` = 0 |
| WR_RESP | Chờ `bvalid` (`bready` = 1); `bresp` ≠ 0 → `err` ← 1 | WR_NEXT |
| WR_FIN | Phát `o_wr_error` nếu `err`, ngược lại `o_wr_done` | WR_IDLE |

**Bảng 7.2:** Kênh ghi

| ID | Yêu cầu |
|---|---|
| WRC-01 | `wvalid` không chờ `awready` (DMA-05). Một khi `wvalid` = 1 thì giữ tới bắt tay, vì đầu FIFO chỉ đổi khi bắt tay. `wvalid` được phép về 0 giữa hai beat khi FIFO rỗng |
| WRC-02 | Burst đã vào WR_BURST là đã cam kết: mọi beat W của nó và phản hồi B đều được hoàn tất, kể cả khi `abort_seen` = 1 (DMA-10). Abort chỉ được xét ở WR_NEXT |
| WRC-03 | Credit: `inflight` = 1 khi có yêu cầu đọc SRAM được nhận ở chu kỳ trước. `i_spm_rd_rvalid` đẩy `i_spm_rd_rdata` vào FIFO. FIFO không bao giờ tràn (assertion) |
| WRC-04 | Tối đa 1 burst ghi outstanding: AW của burst sau chỉ phát sau khi nhận B của burst trước (DMA-06) |
| WRC-05 | Độ trễ: `i_wr_start` ở chu kỳ t → `awvalid` và yêu cầu đọc SRAM đầu tiên ở t + 2; beat W đầu tiên sớm nhất ở t + 4; sau đó một beat mỗi chu kỳ khi `wready` = 1 |

# 8 Reset, hiện thực, kiểm chứng

**Bảng 8.1:** Reset

| ID | Yêu cầu |
|---|---|
| RST-D01 | `rst_n` = 0: hai kênh về IDLE; mọi `valid` AXI ra, `o_*_busy`, xung done/error, FIFO về rỗng |
| RST-D02 | `i_soft_clr`: như RST-D01. FSM chỉ phát `soft_clr` khi cả hai kênh đã rảnh (SFT-02), nên soft clear không bao giờ cắt ngang một giao dịch AXI (assertion AS-08) |

**Bảng 8.2:** Hiện thực

| ID | Yêu cầu |
|---|---|
| IMP-01 | SystemVerilog tổng hợp được, không latch; thanh ghi điều khiển có reset, thanh ghi dữ liệu (địa chỉ, FIFO) không bắt buộc reset |
| IMP-02 | Tính min ba số của AG-02 bằng so sánh; $(4096 - a \bmod 4096)/4$ = `1024-a[11:2]` |
| IMP-03 | `MAX_BURST_BEATS` ngoài 1 … 256 bị chặn lúc elaboration |

**Bảng 8.3:** Assertion bắt buộc (SVA)

| ID | Yêu cầu |
|---|---|
| AS-01 | `arvalid` $\wedge$ `¬arready` $\rightarrow$ ở chu kỳ sau `arvalid` = 1 và `araddr`, `arlen` không đổi; tương tự AW, W (cả `wdata`, `wlast`) |
| AS-02 | `arvalid` $\rightarrow$ `arlen` < `MAX_BURST_BEATS` $\wedge$ `araddr[11:2]` + `arlen` $\leq$ 1023; tương tự AW |
| AS-03 | Không có AR mới khi burst đọc trước chưa có `rlast`; không có AW mới khi chưa nhận B của burst trước |
| AS-04 | Số beat W giữa hai lần `wlast` = `awlen` + 1 của burst tương ứng |
| AS-05 | `i_rd_start` $\rightarrow$ `¬o_rd_busy`; mỗi `i_rd_start` có đúng một `o_rd_done` hoặc `o_rd_error`; tương tự kênh ghi |
| AS-06 | Sau khi `abort_seen` = 1, không có `arvalid` hay `awvalid` nào bật lên mới |
| AS-07 | FIFO không tràn, không lấy khi rỗng |
| AS-08 | `i_soft_clr` $\rightarrow$ cả hai kênh ở IDLE |

**Bảng 8.4:** Tài nguyên ước tính

| Hạng mục | Số lượng |
|---|---|
| Bộ cộng 32 bit (địa chỉ burst, địa chỉ đoạn) | 4 (2 mỗi kênh) |
| Bộ nhân 6 $\times$ 6 (độ dài đoạn liên tục) | 2 (hoặc cộng dồn) |
| FIFO ghi | 4 $\times$ 32 bit |
| Thanh ghi trạng thái và tham số | khoảng 2 $\times$ 140 bit |

**KIỂM CHỨNG BẰNG MÔ HÌNH**

`gemm_dma_model.py`: (1) 15 000 lệnh ngẫu nhiên (5 loại, stride liên tục hoặc có hở, `MAX_BURST_BEATS` 1 … 256): chuỗi (địa chỉ DRAM, địa chỉ SPM) sinh bởi bảng lệnh của FSM + AG-01 … AG-04 khớp đúng công thức phần tử của spec hệ thống (bảng 2-3 và công thức stride mục 7.2), không burst nào vượt 4 KiB hay trải qua khoảng hở; (2) 1 500 lệnh kênh đọc và 1 500 lệnh kênh ghi với slave ngẫu nhiên, gồm lỗi RESP, `rlast` sớm hoặc trễ, abort: dữ liệu đúng khi không lỗi, không AR sau abort, không burst nào bị bỏ dở.